

HunarHub

SDE-1 Interview Preparation Guide

A code-grounded answer bank for a new-grad interview

53 questions with ready-to-say answers, SDE-2/SDE-3 follow-ups, essential HLD extensions, and a candid review of resume claims.

How to rehearse	Answer each prompt aloud in 60–90 seconds. Then read the follow-up and answer it before moving on.
What makes this safe	Implemented behavior is separated from reasonable production improvements so you never overclaim in front of an experienced interviewer.
Best final pass	Prioritize the items marked Resume-critical. They correspond to wording in the resume that will invite deeper probing.

Prepared from the September 2026 resume and the actual HunarHub routes, models, README, client structure, and Postman collection.

How to use this guide

Interviewers rarely ask all of these questions in sequence. Begin with your 60-second project narrative, then let their curiosity choose a branch. Your target is not a memorized speech: it is a clear explanation of what you built, why you chose it, what invariant it protects, and what you would improve next.

Topic map

Section	What it prepares you for	Questions
Start here	Opening pitch, scope, ownership, and trade-offs	5
Architecture	System boundaries, deployment, modularity, MongoDB choice	5
API & data	REST design, models, indexes, validation, rating aggregates	9
Auth & security	JWT, bcrypt, RBAC, ownership, uploads, hardening	9
Flows	Product, order, booking, analytics, and frontend integration	7
Scale & HLD	Caching, queues, reliability, observability, larger-scale design	6
CS fundamentals	HTTP, event loop, ACID, indexing, concurrency	7
Honesty checks	Resume wording and credible responses under deep probing	5

Resume-critical rule

When a resume claim is not exactly reflected by the code, do not try to win an argument. State the current implementation, name the gap and risk, then give the next concrete fix and test. That demonstrates judgment and ownership.

Start here

Opening pitch, scope, ownership, and trade-offs

01 Walk me through HunarHub in 60 seconds.

HunarHub is a MERN marketplace for Indian artisans. Buyers can browse products, place orders, and request custom services. Sellers create and manage their own products, see incoming orders, and accept, reject, or complete service requests. Admins approve sellers, manage service categories, and see platform metrics. I built an Express REST API backed by MongoDB/Mongoose, used JWT plus role and ownership checks, and deployed the React client on Vercel and the API on Render. My emphasis was a small but complete backend: explicit domain models, protected mutations, and testable HTTP endpoints.

Likely follow-up	What was your personal contribution?
Answer	I owned the implementation end to end during the capstone: schema and route design, authentication/authorization, API testing in Postman, and deployment configuration. The important part I would highlight is backend ownership; the app still includes a React client so the API could be exercised as a real product.

02 Why this problem and why both products and service bookings?

A generic catalog would not capture how many artisans work: some sell finished goods, while others take custom work such as tailoring, repair, or bulk pottery. I modeled those as two different workflows. An order is a purchase of a concrete product with quantity, price, shipping address, and shipping status. A service request is an appointment-like intent routed to one seller and progresses through pending, accepted, rejected, or completed. Keeping them separate made each lifecycle clearer.

Likely follow-up	Why not represent services as products?
Answer	That is possible for a simple marketplace, but the important attributes and state transitions differ. A service needs requested date, preferred time, location, and seller acceptance before completion. A product order needs a purchasable SKU-like item, quantity, a price snapshot, and fulfillment status. Separate aggregates avoid a sparse, ambiguous schema.

03 What are the three roles and the key permission boundaries?

A buyer can register, browse public data, place orders, write one review per product, and create service requests. A seller can create, update, and delete only their own products; they can view and update only orders and service requests assigned to them. An admin can approve sellers, manage categories, and view platform analytics and all service requests. Role checks are necessary, but for seller-owned resources I also check the document owner against the authenticated user ID.

Likely follow-up	Is RBAC alone sufficient?
Answer	No. RBAC answers whether a seller category of user may update products; it does not answer whether that seller owns this product. The second check is resource-level authorization, sometimes described as ownership or ABAC-like policy. Without it, one seller could edit another seller's listing.

04 What was the hardest part of building it in one month?

The difficult part was turning requirements into consistent boundaries, not writing individual CRUD handlers. I first identified actors and lifecycles, then attached authorization to each mutation. For example, an order stores both buyer and seller references at creation, so seller views do not depend on a later product lookup. I kept the first release intentionally simple, verified endpoint behavior in Postman, and recorded the production-hardening work I would do next.

Likely follow-up	What did you deliberately defer?
Answer	Payments, stock reservation, notification delivery, cloud object storage for uploads, pagination, rate limiting, and automated CI tests. Those are deliberate next-stage concerns, not features I would pretend the prototype already solves.

05 Tell me about a failure or learning from the project.

My main learning was that a successful happy path is not a security boundary. I added a registration test that sends role=admin and made self-service registration allow only buyer or seller; any other role falls back to buyer. I also learned to distinguish role authorization from ownership checks. In a follow-up version, I would add request validation, tests in CI, and transactional stock handling earlier rather than treating them as polish.

Architecture

System boundaries, deployment, modularity, MongoDB choice

06 Describe the high-level architecture.

It is a decoupled three-tier web application. The React/Vite SPA on Vercel calls an Express API on Render over HTTPS. Express handles middleware, route modules, and static development uploads. Mongoose is the data-access layer to MongoDB Atlas. The browser carries the bearer JWT on protected requests. This separation lets the client scale as static assets/CDN delivery and the API/database scale independently.

Likely follow-up	Trace one protected request end to end.
Answer	The browser sends Authorization: Bearer <token>. Express routes it through protect, which verifies the JWT signature and expiry using JWT_SECRET, then fetches the user without the password and assigns req.user. authorize checks the user's role where required. The route validates the resource relationship, calls Mongoose, serializes a safe response, and the client updates its state.

07 Why did you choose a modular Express router structure?

I grouped routes by bounded responsibility: auth, products, orders, service requests, categories, and admin. server.js stays a composition root that applies common middleware and mounts them. It makes permissions and tests easier to locate, avoids one large controller file, and creates a clean path to extracting a module if the system grows.

Likely follow-up	Would you start with microservices?
Answer	No. For this team size and traffic, a modular monolith is lower operational cost and faster to change. The modules already provide logical boundaries. I would consider extraction only after a measured bottleneck or independent scaling/release need, for example a search/indexing pipeline or asynchronous notification worker.

08 Why MongoDB instead of a relational database?

The marketplace has evolving profile fields, product metadata, image arrays, and a document-oriented prototype lifecycle, so MongoDB allowed rapid schema iteration while Mongoose still gave validation and references. It is not because MongoDB makes relationships disappear: I explicitly store ObjectId references and use populate when a response needs related display data. For payments, inventory reservations, and strict multi-row accounting, a relational database or MongoDB transactions would deserve a closer evaluation.

Likely follow-up	What are MongoDB trade-offs here?
Answer	Flexible documents can lead to inconsistent data if validation is weak. Cross-document joins and transactions need more careful design. Regex search can become costly. I would add indexes based on actual query patterns, paginate listings, use an external search engine for advanced discovery, and use sessions/transactions where business invariants span documents.

09 How would you explain the request path as an HLD diagram verbally?

Client -> CDN-hosted React SPA -> HTTPS JSON request -> Render Express API -> authentication/authorization middleware -> route module -> Mongoose -> MongoDB Atlas. The response travels back as JSON. For seller images in this prototype, Multer writes to the API host's local uploads directory and Express exposes it under /uploads. For production, that branch should instead write to object storage through a signed-upload flow.

10 **How would you make the API horizontally scalable?**

Keep API instances stateless: no in-memory session ownership, configuration in environment variables, and authentication based on signed tokens. Put multiple API instances behind a load balancer, move files to object storage, keep shared data in MongoDB/Redis, and use a worker/queue for non-request work. Then any instance can serve any request. JWT revocation is a trade-off: if needed, use short-lived access tokens plus refresh-token/session records or a deny-list.

API & data

REST design, models, indexes, validation, rating aggregates

11 How many endpoints did you build?

The code currently exposes 32 distinct HTTP route handlers including the health check: auth 3, admin 9, categories 3, products 6, orders 4, service requests 6, and health 1. The Postman collection has 36 requests because it also includes separate test/setup/cleanup examples. On my resume I would say 25+ or, better, 32 API routes; I would not conflate request count with unique endpoints.

RESUME-CRITICAL

Be exact here. An experienced interviewer may test this wording against your implementation.

12 How did you design the REST API?

I used nouns for resources and HTTP verbs for intent: POST /auth/register and /auth/login for creation of authentication state, GET /products for a collection, GET/PUT/DELETE /products/:id for a single product, POST /orders to create an order, and action endpoints such as PUT /orders/:id/status where the state transition is the core operation. Protected routes consistently return 401 for missing/invalid authentication, 403 for authenticated but unauthorized access, 404 for absent resources, 400 for known client conflicts, and 500 for unexpected server errors.

Likely follow-up	Is PUT /:id/status really RESTful?
Answer	It is a pragmatic command-shaped endpoint. A more resource-pure variant would PATCH /orders/:id with a constrained status field. The key design property is that the server validates allowed transitions and ownership; the HTTP shape can be refined as the API matures.

13 What are the six collections and their relationships?

User holds identity, role, seller profile fields, and approval state. Product points to its seller. Order points to buyer, product, and seller. Review points to user and product. Category models active service categories. ServiceRequest points to customer, entrepreneur/seller, and category. References keep the main entities independently addressable; API reads populate only the display fields needed by each screen.

Likely follow-up	Why store seller on Order when Product already has seller?
Answer	It makes the order self-contained for seller inbox queries and preserves the recipient if a product changes or is removed. The cost is denormalization: I must treat the seller copied at order creation as intentional historical data and ensure the server, not the client, derives it from the product.

14 Where are the unique indexes?

User.email and Category.name are single-field unique schema constraints. Review has the compound unique index { product: 1, user: 1 }, enforcing at most one review by a user for a product. I would describe that precisely. The project does not have multiple compound unique indexes; saying plural in the resume would overstate it.

Likely follow-up	Why still check existingReview before insert if the index exists?
Answer	The application check provides a friendly response, while the database unique index is the race-safe final constraint. Two simultaneous requests can both pass a read-before-write check, so production code should also translate a duplicate-key error into a 409 Conflict response.

RESUME-CRITICAL

Be exact here. An experienced interviewer may test this wording against your implementation.

15 How do filters and search work for product listing?

GET /products accepts category, minPrice, maxPrice, rating, search, and sort query parameters. It builds a Mongo query: category can use \$in for comma-separated values, price uses \$gte/\$lte, rating uses \$gte, and search uses case-insensitive regex over title, description, and artisanName. Results are populated with a small seller projection and sorted by newest, price, or rating.

Likely follow-up	What breaks at large scale?
Answer	Unanchored case-insensitive regex can scan many documents, and returning the full result set is unbounded. I would add page/limit with a capped limit, indexes for common filters/sorts, text or Atlas Search for discovery, input limits, and a cursor-based pagination strategy where the catalog becomes large.

16 Why use Mongoose instead of the raw MongoDB driver?

Mongoose gives schemas, validators, middleware, instance methods, model APIs, and reference population in a style that keeps this small application readable. For example, the User pre-save hook centralizes password hashing, and the Order status enum is close to the domain model. It is an ODM, not the database itself; queries still compile into MongoDB operations and index design still matters.

17 How does the review rating update work?

After creating a review, the route reads the product's reviews, calculates the mean, rounds it to one decimal place, writes Product.ratings and Product.numReviews, then returns the populated review. The denormalized fields let listing pages sort/filter by rating without computing an aggregate for every product on every read.

Likely follow-up	Is that race-safe?
Answer	Not fully. Concurrent reviews could calculate from different snapshots and leave a stale aggregate. At higher write volume, I would use an atomic aggregation/update strategy, a transaction where appropriate, or maintain sum and count with atomic increments and derive the average. The compound index remains the invariant for one review per user/product.

18 What validation exists, and what would you improve?

Mongoose enforces required fields, enums, numeric bounds, email format, password minimum length, and update validators where runValidators is passed. The current routes rely too much on model validation. I would add a request-validation layer such as Zod/Joi/express-validator before business logic, explicitly validate ObjectId and query values, whitelist mutable fields, use consistent error serialization, and cap payload sizes.

RESUME-CRITICAL Be exact here. An experienced interviewer may test this wording against your implementation.

19 How do you avoid mass assignment?

The safe pattern is to construct an allowlisted object from named fields rather than passing req.body directly into an update. A few current update routes use req.body directly, so that is a real hardening gap. For example, a product update should permit title, description, price, category, images, location, and inStock, not seller, ratings, or arbitrary schema fields.

RESUME-CRITICAL Be exact here. An experienced interviewer may test this wording against your implementation.

Auth & security

JWT, bcrypt, RBAC, ownership, uploads, hardening

20 Explain registration and login step by step.

Registration accepts name, email, password, role, city, and seller profile fields. The route allowlists only buyer and seller for public signup; attempts to submit admin become buyer. `User.create` triggers the pre-save bcrypt hook, and the response returns non-sensitive profile fields plus a JWT. Login selects the password explicitly because it is `select:false` by default, compares with bcrypt, and returns the same public session payload and a new JWT. `GET /auth/me` verifies the token and returns the user without the password.

Likely follow-up	What goes in the JWT?
Answer	Only the user ID in this project, signed with <code>JWT_SECRET</code> , with a 30-day expiry. The API fetches the current user from the database on each protected request, so role changes take effect even though the token only carries an ID. Do not put passwords or sensitive profile data in a JWT; it is signed, not encrypted.

21 Why hash passwords with bcrypt and a 10-round salt?

Passwords should never be reversible or stored as plaintext. bcrypt is deliberately slow and automatically incorporates a random salt, so identical passwords do not produce identical hashes and precomputed rainbow tables are less useful. In this code `bcrypt.genSalt(10)` sets the work factor. On login `bcrypt.compare` hashes/verifies against the stored hash. The appropriate cost should be benchmarked for the deployment so login remains usable while making offline guessing expensive.

Likely follow-up	What is a salt versus a pepper?
Answer	A salt is unique, random per-password data stored with the hash; bcrypt manages it. A pepper is an extra secret shared by the application and kept outside the database, usually in a secret manager. A pepper can add defense in depth but does not replace a slow password hash, rate limiting, MFA, or secure password reset flows.

22 What do protect and authorize each do?

`protect` is authentication middleware: it extracts a Bearer token, verifies it with `JWT_SECRET`, fetches the user without password, sets `req.user`, and rejects invalid/missing tokens with 401. `authorize(...roles)` is authorization middleware: it checks `req.user.role` and returns 403 if the role is not permitted. Routes compose them as `protect, authorize('seller')` or `protect, authorize('admin')`.

Likely follow-up	Why must authorize run after protect?
Answer	<code>authorize</code> depends on <code>req.user</code> , which <code>protect</code> establishes. Reversing them means the role check has no authenticated subject and can fail incorrectly or create a security bug. Middleware order is part of the API's security contract.

23 How do you prevent privilege escalation at signup?

The server treats role as authority-granting input, so it does not trust arbitrary client data. It uses a server-side allowlist of buyer and seller; a submitted admin role falls back to buyer. Admin accounts are provisioned out of band, for example through a controlled seed/admin path. I also included a Postman request that attempts this escalation so the behavior is demonstrable.

24 Explain seller approval accurately.

A new seller has `isApproved=false`. An admin can list pending sellers and set that flag true on approve or false on reject. That is a pending -> approved or pending -> rejected lifecycle in the current implementation. The seller role gates seller endpoints, but the current product/order/service routes do not also require `isApproved=true`. If approval must block marketplace activity, I would add a reusable `approvedSeller` middleware or enforce that predicate at each seller mutation.

Likely follow-up	Your resume says four-stage workflow. Is that implemented?
Answer	Not as a four-state finite-state workflow. The interview-safe answer is that the code implements registration/pending plus an admin approval decision, with approved/rejected outcomes. I would change the resume to 'seller approval lifecycle' or explicitly implement a four-state enum with transition validation before claiming four stages.

RESUME-CRITICAL

Be exact here. An experienced interviewer may test this wording against your implementation.

25 How do you protect against IDOR/BOLA?

After authenticating and checking the role, routes compare the resource's stored seller/entrepreneur ID with `req.user._id` before allowing a product update/delete, order status update, or service-request transition. That stops a seller from guessing another resource ID and changing it. For buyer-owned reads and mutations, I would apply the same ownership pattern consistently and test cross-user access explicitly.

26 Where should the token live in a browser?

This prototype's frontend uses browser-side auth state; a production decision depends on the threat model. `localStorage` is convenient but exposed to XSS. An `httpOnly`, Secure, SameSite cookie protects the token from JavaScript but introduces CSRF considerations. A robust approach is short-lived access tokens, refresh tokens in secure `httpOnly` cookies, CSP/XSS hardening, CSRF protection for cookie-authenticated mutations, and clear expiry/logout behavior.

27 What security issues would you prioritize before real users?

I would restrict CORS to known origins instead of `app.use(cors())` broadly; add rate limiting to login/register and sensitive APIs; validate/sanitize every request; use a centralized error handler; stop accepting arbitrary update bodies; enforce seller approval; store uploads in object storage with file-size limits and content validation; add security headers/CSP; rotate secrets; and add monitoring/audit logs. Payments and inventory require their own secure design rather than a client-trusted total.

RESUME-CRITICAL

Be exact here. An experienced interviewer may test this wording against your implementation.

28 How does image upload work and what are its risks?

The product route uses Multer disk storage, accepts up to five images, checks extension and MIME type against `jpeg/jpg/png/webp`, saves files under `server/uploads`, and exposes that directory through Express. It is fine for a prototype, but extension/MIME checks are not sufficient security. I would enforce file size/count, inspect content, generate server-side filenames, scan if required, serve from object storage/CDN, and avoid relying on `Render local disk` for durable uploads.

Flows

Product, order, booking, analytics, and frontend integration

29 Walk through product creation and ownership enforcement.

A seller sends a protected multipart POST /products. protect authenticates, authorize('seller') gates the role, Multer handles up to five uploaded images, and the server derives seller from req.user._id rather than trusting the body. Mongoose validates the product and stores it. For PUT/DELETE, the route first loads the product and compares product.seller to req.user._id. Only then does it modify/delete it.

Likely follow-up	What improvement would you make to deletion?
Answer	Decide the lifecycle policy. Deleting a product that has orders can leave historical references awkward. I would either preserve immutable order snapshots and soft-delete/hide the listing, or reject deletion when active orders exist. Images should also be deleted asynchronously from object storage only after that policy is safe.

30 Walk through order placement.

The buyer posts productId, quantity, and shipping address. The server loads the product, returns 404 if absent, derives seller from product.seller, calculates totalPrice on the server as product.price times quantity, creates the order with buyer from req.user, then populates product and seller fields for the response. The seller later reads orders where seller equals their own ID and can change the fulfillment status only if they own the order.

Likely follow-up	What is wrong with this at checkout scale?
Answer	There is no inventory decrement/reservation, no price snapshot field distinct from mutable product price, no payment provider verification, no allowed-transition validation, and no transaction/idempotency key. I would validate quantity, create a price snapshot, reserve/decrement stock atomically in a transaction or conditional update, process payment via a provider webhook, and make retries idempotent.

31 How would you make order creation idempotent?

A client retry must not create two orders. I would require an Idempotency-Key for POST /orders, store it with buyer ID and a request fingerprint under a unique compound index, and in a transaction return the previously completed response for the same key. The key should have a suitable retention period. Payment provider idempotency must be coordinated with the same business operation.

32 Walk through a service booking.

A customer first reads active categories and approved sellers, then POSTs a protected request with entrepreneurId, categoryId, description, requested date/time, and location. The API validates that the selected user is a seller, creates a pending ServiceRequest linking customer, entrepreneur, and category, and populates display fields. The assigned seller sees incoming requests and may accept, reject, or complete only their own request. Admins can view all requests.

Likely follow-up	What needs tightening?
Answer	The category-to-seller query currently verifies the category but returns all approved sellers rather than matching their skills; it should model category skills or a SellerCategory join/array and query it. I would also validate allowed state transitions, prevent a seller from completing a pending request, and ensure categoryId refers to an active category.

33 What is the order status state machine?

The model enumerates pending, shipped, delivered, and cancelled, with pending as default. The current implementation checks seller ownership but accepts the requested status directly. A production state machine should define allowed edges, for example pending -> shipped/cancelled and shipped -> delivered, reject backward/invalid transitions, capture timestamps and actor IDs, and make terminal states immutable except through a supervised exception flow.

34 How are admin analytics calculated?

The admin-only analytics route counts buyer users, seller users, unapproved sellers, orders, and service requests. It runs an Order.aggregate pipeline with \$group and \$sum over totalPrice to calculate total revenue. Doing the sum in MongoDB avoids loading every order into Node.js and lets the database do set-based aggregation.

Likely follow-up	Is that revenue definition correct?
Answer	It is gross order value across all orders in the current code. It includes pending and cancelled orders because no status filter is applied, and does not account for refunds, fees, taxes, or payment settlement. I would label it 'gross order value' or filter to paid/delivered depending on the business definition.

35 How does the frontend integrate with the backend?

The React client uses React Router for pages, Axios for API requests, AuthContext for session/auth state, and CartContext with local-storage persistence for cart UX. VITE_API_URL points the deployed SPA to the /api base of the Render service. The backend is not dependent on Vercel; that decoupling lets either client or API deploy independently.

Scale & HLD

Caching, queues, reliability, observability, larger-scale design

36 Design HunarHub for 1 million users. What changes first?

Keep the core service a modular monolith initially, but separate responsibilities operationally: CDN for static assets/images, load-balanced stateless API instances, MongoDB replica set with indexes/read replicas as needed, Redis for hot catalog/cache/rate limiting, object storage for uploads, a queue plus workers for notifications/image processing, and observability. Add pagination and a search service before catalog queries become expensive. Payments/orders remain a strongly controlled transactional path; analytics should move toward event-driven aggregates or a warehouse as reporting grows.

Likely follow-up	Where are the consistency boundaries?
Answer	Authentication and catalog reads can be eventually consistent in many places. Checkout/payment, inventory reservation, order status, refunds, and permission-changing admin operations require stronger invariants, transactions/conditional updates, audit trails, and idempotency. I would make those boundaries explicit instead of trying to make every feature globally strongly consistent.

37 What would you cache?

Public, read-heavy, slowly changing data: category list, popular product pages, filtered listing pages with carefully chosen keys, and seller public profiles. I would use cache-aside with TTLs and invalidate/update relevant keys when a product/category changes. I would not casually cache authorization decisions or mutable checkout state without clear invalidation rules.

Likely follow-up	What cache problems are you watching for?
Answer	Stale data, cache stampedes, unbounded key cardinality from arbitrary search parameters, and invalidation bugs. I would normalize cache keys, cap pagination/filter combinations, use TTL plus targeted invalidation, and let the database remain the source of truth.

38 How would you add notifications?

The synchronous booking/order write would commit first. It would then emit an outbox event in the same transaction or persist an event record; a worker consumes it and sends email/push/in-app notifications with retry and dead-letter handling. This avoids making the user's successful API request depend on an external mail provider and prevents lost events from a crash between DB write and publish.

39 How would you handle a Render/API outage?

Health checks and uptime monitoring should detect it; logs, metrics, and traces should identify whether the API, MongoDB, or dependency failed. For recovery, run multiple instances/regions as the need justifies, use managed database backups and restore drills, place assets on CDN/object storage, and maintain rollbackable versioned deployments. User-facing behavior should return clear retry-safe errors, especially for idempotent checkout operations.

40 What metrics would you monitor?

Golden signals: request rate, error rate by status/route, latency percentiles, and saturation for API/DB. Then domain metrics: registration/login failures, seller approval backlog, product search latency, order-creation success and duplicate-key/idempotency outcomes, booking acceptance rate, queue depth, and payment webhook failures. Alerts should be tied to user impact and runbooks, not every raw log line.

41 **How would you evolve analytics?**

The live aggregation is good for a small admin dashboard. At scale, running expensive aggregates against the transactional collection can harm user traffic. I would publish order lifecycle events to a queue/stream, build daily/hourly materialized aggregates in a reporting store, and define revenue precisely. The operational dashboard can remain near-real-time, while finance uses reconciled immutable records.

CS fundamentals

HTTP, event loop, ACID, indexing, concurrency

42 JWT versus server session: what is the trade-off?

A server session uses an opaque cookie that maps to server-side session state; revocation and forced logout are straightforward but require a shared session store when instances scale. A JWT is self-contained and can be verified by any instance, reducing per-request session lookup, but expiry/revocation and token leakage need more design. This project uses JWT with user ID and database lookup for the current user. Neither is automatically more secure; storage, transport, expiry, and XSS/CSRF defenses matter.

43 What is the Node.js event loop and why does it matter here?

Node.js runs JavaScript callbacks on an event loop and uses non-blocking I/O so it can handle many concurrent network/database waits without one thread per request. Express route handlers await Mongoose/bcrypt operations instead of blocking. CPU-heavy work such as image transformation, huge reports, or expensive synchronous crypto can block the event loop, so I would send it to worker threads or background workers.

44 Explain HTTP status codes you used.

201 Created for successful registration/product/order/review/service-request creation; 200 for successful reads/updates; 400 for known client errors such as an existing user/review; 401 when no valid authentication is present; 403 when authentication succeeds but policy/ownership fails; 404 for a missing resource; and 500 for unexpected server failures. For duplicate writes backed by a database unique constraint, I would prefer 409 Conflict.

45 PUT, PATCH, POST, and DELETE: how do you choose?

POST creates a subordinate resource or invokes a non-idempotent command; GET reads and should be safe; PUT conventionally replaces a known resource and should be idempotent; PATCH applies a partial update; DELETE removes/hides a resource and is intended to be idempotent from the caller perspective. In a small API, conventions can be pragmatic, but I keep behavior documented and validate state transitions on the server.

46 What is an index and why is it not always good?

An index is an auxiliary data structure that speeds reads/filtering/sorting by avoiding a full collection scan. It consumes storage and slows writes because each insert/update must maintain it. I start from measured query patterns: email login and category uniqueness benefit from indexes; the review compound index both enforces a business rule and supports product-plus-user lookup. I verify with explain plans rather than indexing every field.

47 What does ACID mean, and where does it matter here?

Atomicity means a multi-step operation completes fully or not at all; consistency preserves invariants; isolation limits interference between concurrent operations; durability survives failures. It matters most for inventory, payment/order creation, refunds, and approval audit records. The current prototype has simple independent writes; a production checkout would use conditional updates or a MongoDB transaction and an idempotency design to uphold those invariants.

48 What is a race condition in this project?

Two buyers can see the same stock and both order it before either write reserves inventory; the current code has no stock reservation. Two review requests can both pass an application-level 'no existing review' check; the compound unique index saves the data invariant but the error response needs handling. Concurrent rating recomputations can produce a stale derived average. The fixes differ: atomic conditional updates/transactions for stock, database constraints plus 409 handling for review uniqueness, and atomic aggregates/events for rating data.

Honesty checks

Resume wording and credible responses under deep probing

49 Can I say 'automated CI/CD on Render' based on this repository?

Say this carefully: 'GitHub-triggered continuous deployment on Render and Vercel; each push to main triggers a build and deployment.' The repository has no GitHub Actions/workflow file, automated test runner, or Render blueprint, so 'CI/CD' can invite a question about test gates you cannot show. If asked about next steps: add lint/unit/integration/Postman CLI tests in GitHub Actions, require checks before main, deploy to staging, smoke-test, then promote/rollback.

RESUME-CRITICAL

Be exact here. An experienced interviewer may test this wording against your implementation.

50 Can I say 'cut manual release steps to 0'?

You can say it only in the narrow deployment sense: after the Render/Vercel projects and environment variables were configured, a push to main automatically built and deployed both services, so there was no manual artifact upload or server SSH release step. Do not imply automated test approval, migration orchestration, or production rollback automation that you did not implement.

RESUME-CRITICAL

Be exact here. An experienced interviewer may test this wording against your implementation.

51 How should I answer 'validated all endpoints with Postman' ?

I created a Postman collection organized by health, auth, admin, categories, products, orders, and service requests. It contains happy paths and important authorization checks, including a public registration privilege-escalation attempt. I used collection variables/tokens and manually verified status/payloads. Be precise: the collection is test coverage evidence, not a full automated CI integration suite unless you actually run it through Newman in CI.

52 What resume corrections make this stronger and safer?

Replace '25+ endpoints' with '32 REST API routes' if you want precision. Replace 'compound unique indexes' with 'a compound unique index on review (product, user), plus unique constraints on email and category name.' Replace '4-stage seller approval workflow' with 'seller approval lifecycle (pending, approved, rejected)' unless you implement four explicit states. Replace 'automated CI/CD' with 'GitHub-triggered continuous deployment on Render/Vercel' unless a real CI workflow is added. These edits make your story more credible under deep questioning.

RESUME-CRITICAL

Be exact here. An experienced interviewer may test this wording against your implementation.

53 If an interviewer finds a gap, what is the best response?

Do not defend an inaccurate claim. Say: 'You're right; in the first production prototype I implemented X, but Y is not yet enforced. The risk is Z. My next change would be A because it protects the invariant, and I would test it with B.' That response demonstrates judgment, ownership, and ability to learn - qualities an SDE-1 interviewer values more than pretending the code is enterprise-complete.